

GBM8378 - Principes d'imagerie biomédicale

Laboratoire 1 - Computed Tomography (Tomodensitométrie)

Zire Aymen - 2301311

Rabhi Ayoub - 2131345

Assurez vous d'avoir indiqué vos noms et matricules dans la cellule précédente.

Avant de commencer à répondre aux différentes questions, veuillez vérifier que le Jupyter Notebook fonctionne correctement:

1. Redémarrez le noyau (dans la barre de menu : "Kernel" → "Restart")
2. Lancez toutes les cellules (dans la barre de menu : "Cell" → "Run All").

Travail à effectuer

[Le PDF contenant la partie théorique du TP est disponible sur Moodle.](#)

Avant de remettre votre travail, assurez-vous d'avoir rempli toutes les sections
Commencez votre code ici et Double-cliquez pour entrer votre réponse.

Section 1 - Questions préliminaires (5pts)

1.1. Quel est l'unité de μ , le coefficient d'atténuation des rayons X?

Dans le système international, l'unité du coefficient d'atténuation linéaire μ est le mètre inverse (m^{-1}), parce que le produit dans la loi de Beer-Lambert (loi d'atténuation des rayons X) du coefficient d'atténuation par la largeur du tissu, soit $\mu \cdot \Delta x$, doit être sans dimension dans l'exponentielle pour respecter les unités.

1.2. L'équation (1) de la partie théorique fait l'hypothèse d'une source de photons monochromatique. Peut-on considérer que chaque photon conserve la même énergie tout au long de sa propagation dans l'objet ? Justifier.

Non, dans l'objet, une partie des photons interagit par diffusion Compton et certains sont absorbés par l'effet photoélectrique et perdent de l'énergie, donc l'énergie des photons transmis n'est pas forcément la même que celle des photons incidents. L'équation (1) suppose plutôt qu'on ne considère que les photons non diffusés et non absorbés et qu'on néglige le durcissement du faisceau.

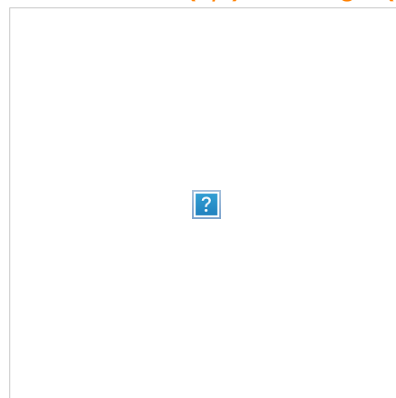
1.3. En pratique les sources à rayons X ne sont pas monochromatiques. Pourquoi ?

Dans un cadre pratique les sources à rayons X ne sont pas monochromatiques car les photons sont principalement produits par rayonnement de Bremsstrahlung lors de la décélération des électrons dans l'anode, ce qui génère un spectre continu d'énergies. À cela s'ajoutent des raies caractéristiques liées au matériau de la cible. L'énergie des photons Bremsstrahlung dépend de la distance d'approche et de l'interaction entre l'électron incident et le noyau, ce qui rend le spectre polychromatique.

1.4. On considère ici que les rayons X se propagent en ligne droite dans l'objet à imager. Comment peut-on limiter la détection de rayons-x issus de la diffusion de Compton pour que cette hypothèse soit vraie ?

On peut limiter la détection des rayons X diffusés en utilisant une collimation du faisceau afin de restreindre le champ irradié et réduire la diffusion (réduire effet broadband), ainsi que des grilles anti-diffusantes placées devant le détecteur, qui absorbent les photons déviés par la diffusion Compton et laissent principalement passer les photons se propageant quasi rectilignement vers le détecteur.

1.5. À partir de la figure 4 de la partie théorique, décrivez qualitativement à quoi ressemblerait la transformée de Fourier $F(u,v)$ de l'image $f(x,y)$ dans le cas suivant :



Comme l'image $f(x,y)$ varie principalement selon x (bandes verticales) et est quasi constante selon y , le spectre $|F(u,v)|$ est concentré sur l'axe $v = 0$. On pourrait observer un pic central (composante DC) si le signal est positif et des pics symétriques à $\pm u_0$ correspondant à la fréquence spatiale des bandes. Les pics devraient être proches du centre étant donné que la fréquence de l'image est basse.

Section 2 - Projection et rétroprojection (2pts)

Simuler la projection d'une image fantôme et calculer sa rétroprojection.

`sino_ES_S` contient le sinogramme. Celui-ci a été réalisé à partir de 180 projections acquises pour $\theta = [0 \ 179]^\circ$.

```
In [17]: # Dans cette cellule, nous importons les librairies/fonctions qui vous seront

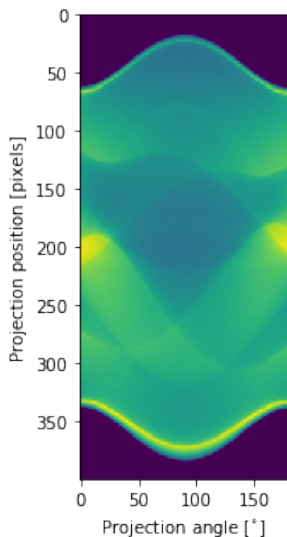
# Librairies
import numpy as np
import matplotlib.pyplot as plt
import skimage.data

# Fonctions
from scipy.ndimage import convolve
from scipy.ndimage.interpolation import rotate
from numpy.fft import fft, fft2, ifft, fftshift, ifftshift
from skimage.transform import resize, radon, iradon

%matplotlib inline
```

```
In [5]: # Créer le sinogramme à partir du fantôme de Shepp-Logan de skimage.data
sino_ES_S = radon(skimage.data.shepp_logan_phantom()) # ES_S = espace spatia

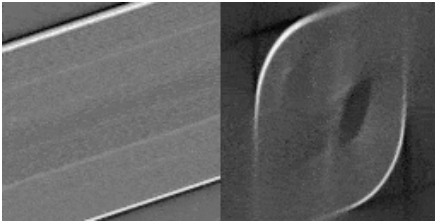
# Afficher le sinogramme
plt.figure(figsize=[5, 5])
plt.imshow(sino_ES_S)
plt.xlabel('Projection angle [ $^\circ$ ]')
plt.ylabel('Projection position [pixels]');
```



2.1. Effectuer la rétroprojection du sinogramme à l'aide de la fonction `skimage.transform.iradon` précédemment importée. (1pt)

Attention, vous ne devez pas effectuer de filtrage lors de la rétroprojection. Pour cela, référez-vous à la documentation de la fonction `iradon`.

L'animation ci-dessous, produite par l'Ecole Polytechnique Fédérale de Lausanne, montre comment la rétroprojection est obtenue à partir du sinogramme:



- Chaque projection contenue dans le sinogramme est étirée sur la largeur de l'image désirée. - Toutes ces projections sont ajoutées les unes aux autres en les rotant aux angles auxquels elles ont été respectivement acquises.

In [7]:

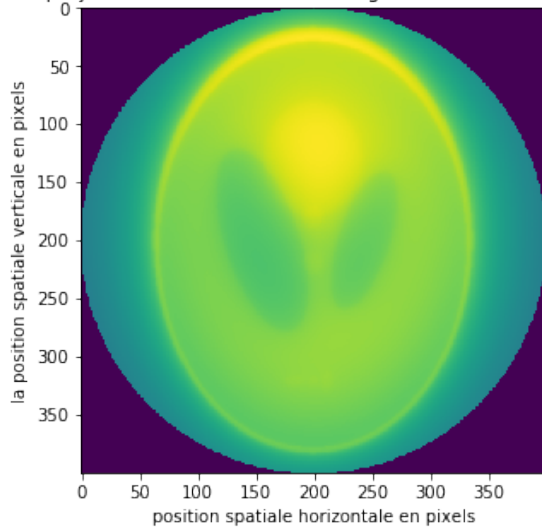
```
angles = range(180)

# Calculez et affichez la rétroprojection.
# >>>> Commencez votre code ici <<<<<
recon_bp = iradon(sino_ES_S, theta=angles, filter_name=None) #calcul de la ré

#générer la figure
plt.figure(figsize=[5, 5])
plt.imshow(recon_bp)
plt.title('Rétroprojection sans utiliser de filtrage dans la fonction iradon')
plt.axis('on')
plt.xlabel('position spatiale horizontale en pixels')
plt.ylabel('la position spatiale verticale en pixels')
```

```
Out[7]: Text(0, 0.5, 'la position spatiale verticale en pixels')
```

Rétroprojection sans utiliser de filtrage dans la fonction iradon



2.2. Pourquoi le sinogramme a-t-il été acquis sur 180° plutôt que sur 360° ? (1pt)

Le sinogramme n'a besoin d'être acquis que sur 180° car les projections à $\theta + 180^\circ$ sont redondantes et contiennent la même information que celles à θ pour une géométrie à rayons parallèles. Si on aurait été jusqu'à 360° on aurait obtenues une superposition l'image actuelle deux fois.

Section 3 - Rétroprojection filtrée (5pts)

L'objet que vous devriez visualiser est un fantôme de Shepp-Logan. Il s'agit d'une image fréquemment utilisée pour simuler une tête humaine lors du développement d'algorithmes de reconstruction d'images médicales.

Vous pouvez constater que le centre de l'image est très intense et produit un effet "d'éblouissement". Ce phénomène est lié au processus de rétroprojection et nous allons maintenant voir comment il peut être compensé afin d'obtenir une image plus homogène et fidèle à la réalité. Pour cela, nous allons filtrer notre signal à l'aide d'une rampe.

3.1. Par quelle opération mathématique peut-on appliquer le filtre rampe $|\omega|$ dans le domaine fréquentiel ? (1pt)

Le filtre rampe $|\omega|$ est appliqué par une multiplication dans le domaine fréquentiel entre la transformée de Fourier (1D) de chaque projection et la fonction $|\omega|$. Ce filtre amplifie les hautes fréquences et compense le suréchantillonnage des basses fréquences au centre de l'espace de Fourier, mis en évidence par le théorème de la tranche centrale. Ce qui permet d'effectuer une rétroprojection filtrée et d'améliorer la netteté de l'image reconstruite.

3.2. Quel est l'équivalent de cette opération dans le domaine image (spatial) ? (1pt)

D'après le théorème de Fourier, cette multiplication fréquentielle est équivalente à une convolution dans le domaine spatial.

3.3. Appliquez les filtres suivants à votre sinogramme et comparez les

reconstructions obtenues: - aucun filtre

- un filtre $|\omega|$ dans le domaine fréquentiel shifté

- un filtre $|\omega|$ dans le domaine fréquentiel non-shifté

- un filtre $|\omega|$ dans le domaine spatial shifté

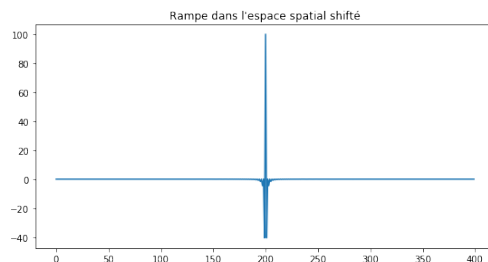
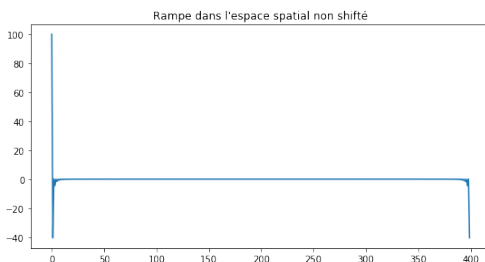
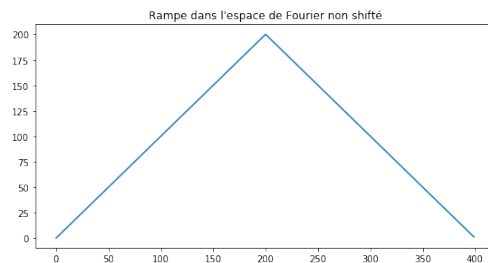
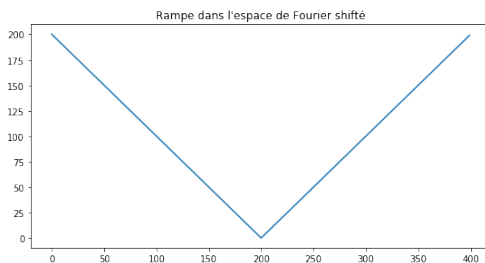
Ci-dessous, le filtre $|\omega|$ vous est présenté dans chacun des espaces d'intérêt.

Sélectionnez le.s filtres appropriés pour vos calculs et affichez les rétroprojections filtrées ainsi obtenues. (2pt)

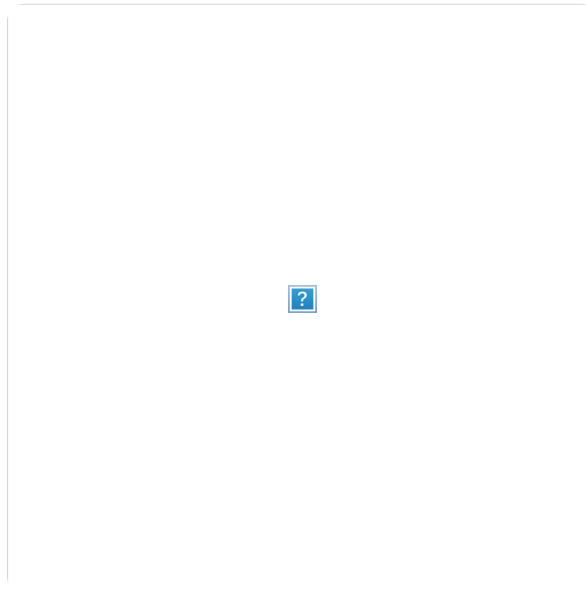
```
In [13]: # Construction et affichage des filtres que vous pourrez utiliser:

nb_pixels = sino_ES_S.shape[0]
# Rampe dans l'espace de Fourier shifté (en 1D, basses fréquences au centre):
ramp_EF_S = np.asarray([abs(nb_pixels / 2 - i) for i in range(nb_pixels)]).T
# Rampe dans l'espace de Fourier non shifté (en 1D, basses fréquences sur les
ramp_EF_NS = ifftshift(ramp_EF_S)
# Rampe dans l'espace spatial non shifté:
ramp_ES_NS = np.real(ifft(ramp_EF_NS))
# Rampe dans l'espace spatial shifté:
ramp_ES_S = fftshift(ramp_ES_NS)

plt.figure(figsize=[20, 10])
plt.subplot(2, 2, 1)
plt.plot(ramp_EF_S)
plt.title('Rampe dans l\'espace de Fourier shifté')
plt.subplot(2, 2, 2)
plt.plot(ramp_EF_NS)
plt.title('Rampe dans l\'espace de Fourier non shifté')
plt.subplot(2, 2, 3)
plt.plot(ramp_ES_NS)
plt.title('Rampe dans l\'espace spatial non shifté')
plt.subplot(2, 2, 4)
plt.plot(ramp_ES_S)
plt.title('Rampe dans l\'espace spatial shifté');
```



Différents espaces relatifs aux transformées de Fourier



Les rappels suivant vous aideront à manipuler vos données entre les domaines spatiaux et fréquentiels:

* Passer directement de l'espace Spatial shifté à l'espace de Fourier non shifté ne conserve pas la symétrie et l'aspect réel pur des fonctions. * Cette erreur est généralement mineure lorsque l'on affiche la valeur absolue du résultat. Cependant, des erreurs visibles arrivent lorsqu'on fait des opérations sur des résultats erronés, par exemple des filtrages. * Pour éviter toute erreur, il faut simplement faire nos opérations mathématiques avec des objets (matrices, vecteurs..) qui sont dans le même espace.

In [14]:

```

# ==== Filtrage dans l'Espace Fourier shifté (4 lignes max) ====>

# >>>> Commencez votre code ici <<<<
# calcule la FFT du sinogramme puis centre les fréquences avec fftshift ensuite
# On décentre (ifftshift) puis on fait l'iFFT pour obtenir le sinogramme filtré

S = fftshift(fft(sino_ES_S, axis=0), axes=0)
sino_f = np.real(ifft(ifftshift(S * ramp_EF_S[:, None], axes=0), axis=0))
retro_EF_S = iradon(sino_f, theta=angles, filter_name=None)

# ==== Filtrage dans l'Espace Fourier non shifté (4 lignes max) ====>

# >>>> Commencez votre code ici <<<<

# applique FFT du sinogramme dans l'ordre naturel (non shifté).
# Le spectre est multiplié par la rampe ramp_EF_NS. L'iFFT permet de revenir à l'ordre naturel.

S = fft(sino_ES_S, axis=0)
sino_f = np.real(ifft(S * ramp_EF_NS[:, None], axis=0))
retro_EF_NS = iradon(sino_f, theta=angles, filter_name=None)

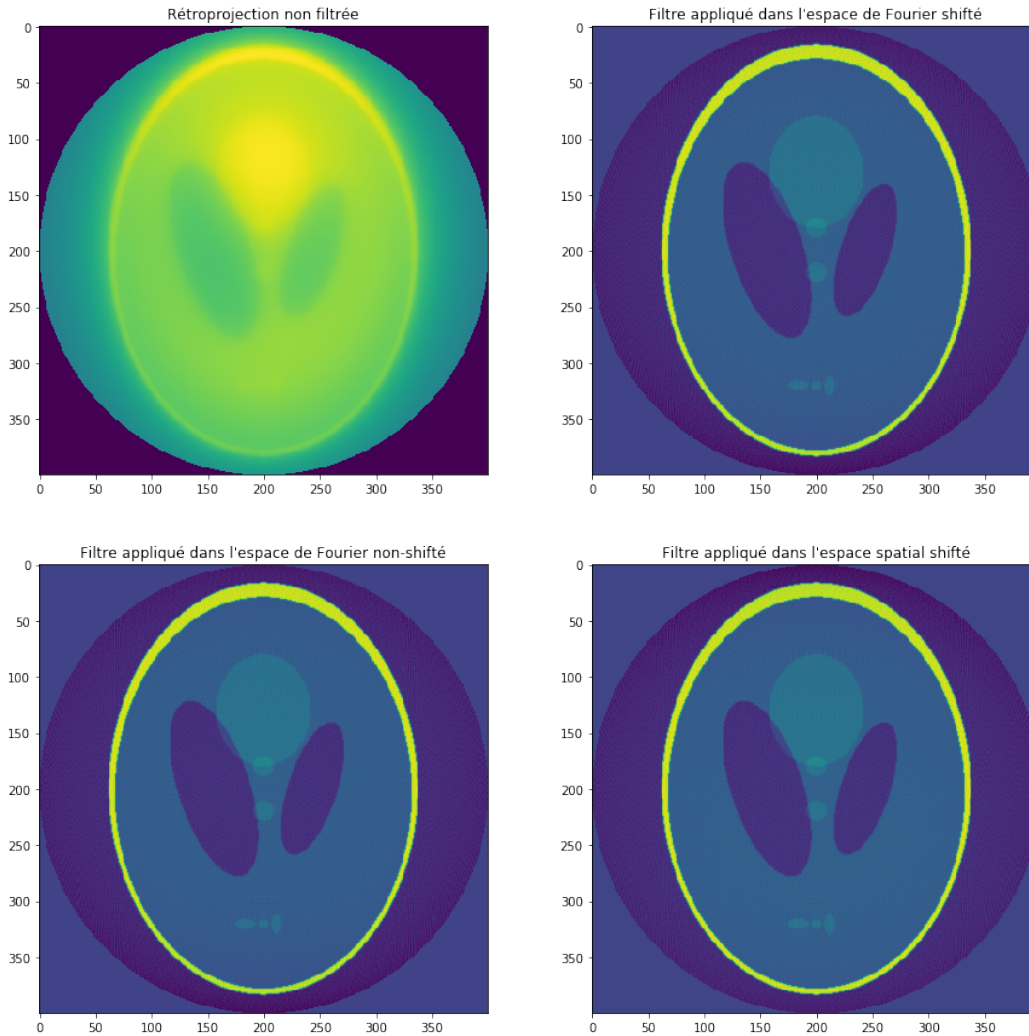
# ==== Filtrage dans l'Espace Spatial shifté (4 lignes max) ====>

# >>>> Commencez votre code ici <<<<

# On filtre directement le sinogramme par convolution avec la rampe ramp_ES_S
# le long de l'axe détecteur (filtrage spatial équivalent au fréquentiel). Le
sino_f = convolve(sino_ES_S, ramp_ES_S[:, None], mode='reflect')
retro_ES_S = iradon(sino_f, theta=angles, filter_name=None)

# Affichage des images reconstruites:
plt.figure(figsize=[15, 15])
plt.subplot(221); plt.imshow(iradon(sino_ES_S, angles, filter_name=None)); plt.title('Image reconstruite sans filtre')
plt.subplot(222); plt.imshow(retro_EF_S); plt.title('Filtre appliqué dans l\'Espace Fourier shifté')
plt.subplot(223); plt.imshow(retro_EF_NS); plt.title('Filtre appliqué dans l\'Espace Fourier non shifté')
plt.subplot(224); plt.imshow(retro_ES_S); plt.title('Filtre appliqué dans l\'Espace Spatial')

```

3.4. Expliquer l'intérêt du filtre $|\omega|$ dans le cadre d'une rétroprojection par rapport à d'autres types de filtre. L'espace d'application du filtre (spatial ou fréquentiel) a-t-il un effet sur les images filtrées? (1pt)

Le filtre rampe $|\omega|$ sert à enlever le flou créé par la rétroprojection. En amplifiant les hautes fréquences, il renforce les détails et les contours, mais il peut aussi augmenter le bruit. D'un autre côté d'autres filtres réduisent ce bruit, mais rendent l'image plus lisse. Appliquer le filtre en spatial ou en fréquentiel donne le même résultat en théorie, car ce sont deux façons équivalentes de filtrer. En pratique, on préfère le fréquentiel car c'est plus simple numériquement et plus rapide à interpréter.

Section 4 - Performance de la rétroprojection filtrée (2pts)

4.1. Comparer les rétroprojections filtrées effectuées à partir de 4, 16, 90 et 180 angles régulièrement espacés (1pt)

Vous pouvez utiliser le filtrage de la fonction iradon.

```

# Performance de la rétroprojection filtrée
n_angles=[4, 16, 90, 180];

plt.figure(figsize = [15, 15])
for n in range(4):
    # >>>> Commencez votre code ici <<<<<
    #réimporter le fantôme
    image = shepp_logan_phantom()

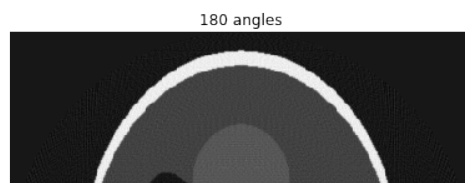
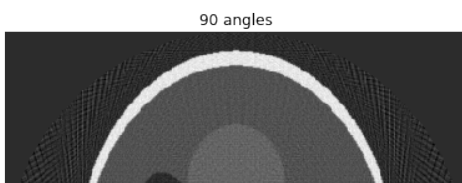
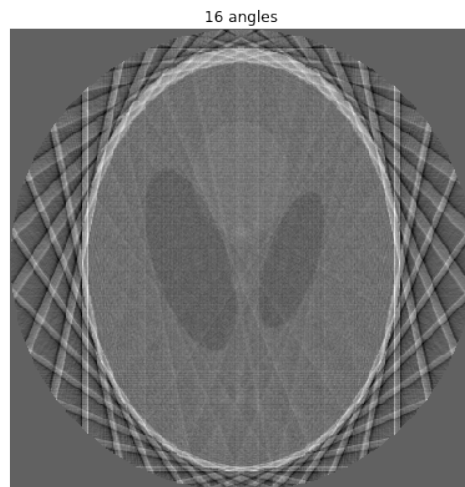
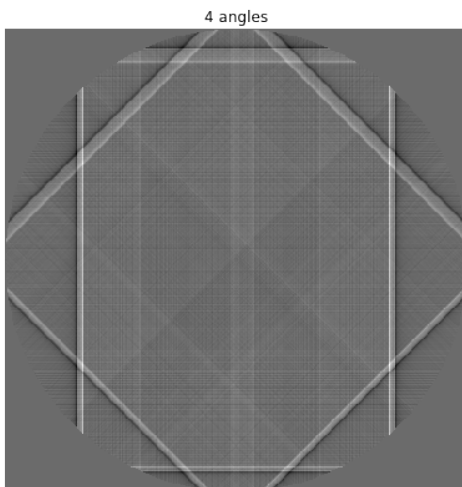
    nb = n_angles[n]    #prendre l'angle associé
    theta = np.linspace(0, 180, nb, endpoint=False) #créer le vecteur selon n

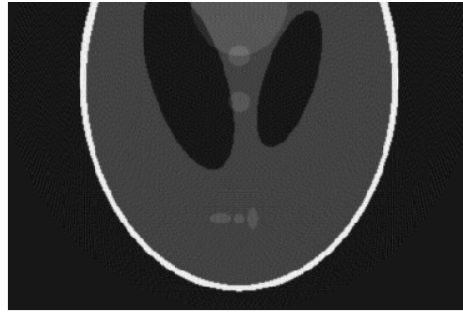
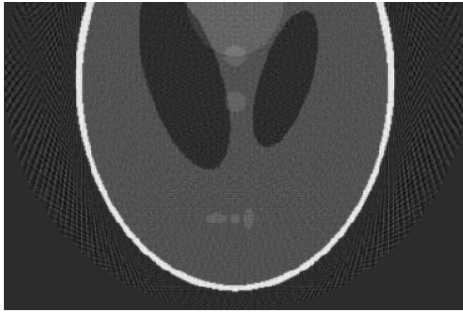
    sinogram = radon(image, theta=theta)
    reconstruction = iradon(sinogram, theta=theta, filter_name='ramp') #rétro

    #construire la figure de rétroprojections filtré
    plt.subplot(2,2,n+1)
    plt.imshow(reconstruction, cmap='gray')
    plt.title(str(nb) + " angles régulièrement espacés")
    plt.axis('off')

plt.show()

```





4.2. Quel serait l'avantage d'utiliser moins d'angles? (0.5pt)

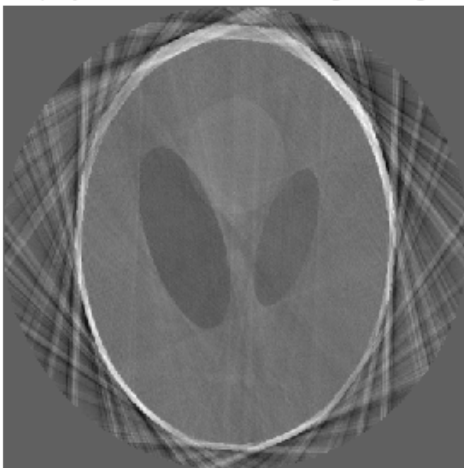
En utilisant moins d'angles, on réduit le temps de calcul et l'utilisation de la mémoire. Cela diminue aussi la dose de rayons X, qui sont ionisants et peuvent être dangereux à long terme pour le patient.

4.3. Qu'arrive-t-il si les rétroprojections sont effectuées pour des angles qui ne sont pas régulièrement espacés ? Illustrez ce phénomène. (0.5pt) Utilisez environ 60 angles différents.

In [25]:

```
n_angles = 60
#on génère 60 valeurs différentes entre 0 et 1 et on multiplie par 180 pour a
theta_irreg = np.sort(np.random.rand(n_angles) * 180)
#générer le sinogramme
sinogram = radon(image, theta=theta_irreg)
#rétroprojection filtrée avec angle irrégulier
reconstruction = iradon(sinogram, theta=theta_irreg, filter_name='ramp')
#générer la figure
plt.figure(figsize=(5,5))
plt.imshow(reconstruction, cmap='gray')
plt.title("Rétroprojection filtrée avec 60 angles irréguliers")
plt.axis('off')
plt.show()
```

Rétroprojection filtrée avec 60 angles irréguliers



Certaines directions de l'image sont mal couvertes, ce qui donne un effet de flou et des artefacts directionnels sous forme de stries, car l'image est sous-échantillonnée par endroit. Cependant l'image est reconnaissable mais la reconstruction est dégradée.

Section 5 - Théorème de la tranche centrale (3pts)

Montrez que la transformée de Fourier 1D de la projection de l'image pour un angle θ est l'équivalent de la tranche d'angle θ de la transformée de Fourier 2D de la rétroprojection $f(x, y)$.

Pour cela, écrivez un script permettant de les afficher sur un même graphe. Faire la comparaison pour 4 angles différents. Utiliser une échelle logarithmique pour mieux visualiser vos résultats. Expliquez. (3pt)

Astuce: la fonction [`scipy.ndimage.interpolation.rotate`]

([https://docs.scipy.org/doc/scipy-](https://docs.scipy.org/doc/scipy-0.14.0/reference/generated/scipy.ndimage.interpolation.rotate.html)

[0.14.0/reference/generated/scipy.ndimage.interpolation.rotate.html](https://docs.scipy.org/doc/scipy-0.14.0/reference/generated/scipy.ndimage.interpolation.rotate.html)) permet de rotater une image d'un certain angle.

In [27]:

```

F2D = fftshift(fft2(image))
F2D_log = np.log(np.abs(F2D)+1) # Passage en échelle logarithmique pour mieu

angles = [0, 170, 60, 90]
plt.figure(figsize=(12,8))

for i, theta in enumerate(angles):

    img_rot = rotate(image, theta, reshape=False) # Rotation de l'image pour s
    projection = np.sum(img_rot, axis=0) # Projection = intégration le long d'

    F1D = fftshift(fft(projection)) #TF
    F1D_log = np.log(np.abs(F1D)+1)

    N = image.shape[0] # Paramètres pour extraire une tranche radiale dans la
    center = N//2
    t = np.linspace(-center, center-1, N)

    # Coordonnées de la tranche à l'orientation  $\theta$ 
    u = center + t*np.cos(np.deg2rad(theta))
    v = center + t*np.sin(np.deg2rad(theta))
    u = np.clip(u.astype(int), 0, N-1) #conversion
    v = np.clip(v.astype(int), 0, N-1)

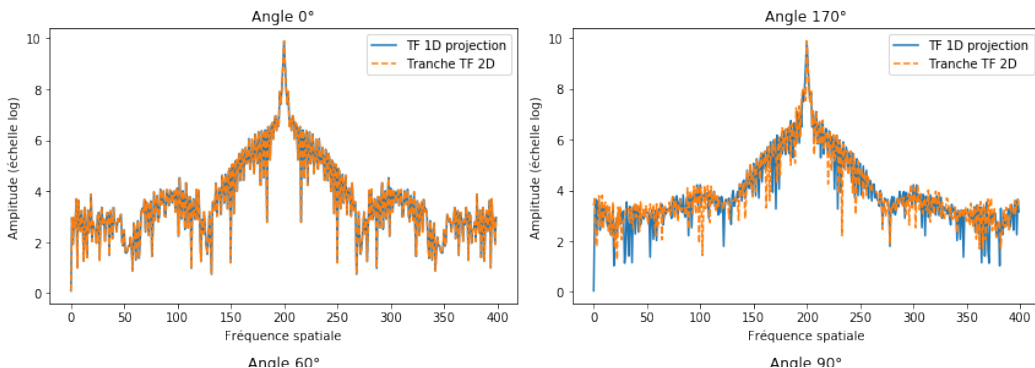
    # Extraction de la tranche correspondante dans la TF 2D
    slice_F2D = F2D_log[v, u]

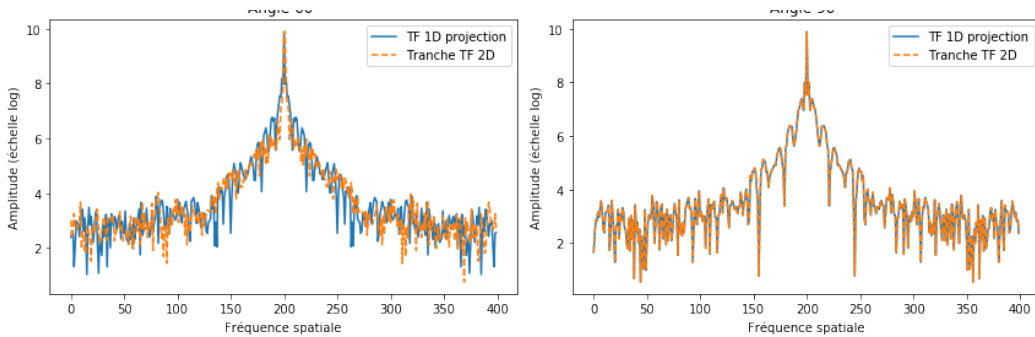
    #générer les figures
    plt.subplot(2,2,i+1)
    plt.plot(F1D_log, label='TF 1D projection')
    plt.plot(slice_F2D, '--', label='Tranche TF 2D')
    plt.title(f'Angle {theta}°')
    plt.xlabel('Fréquence spatiale')
    plt.ylabel('Amplitude (échelle log)')

    plt.legend()

plt.tight_layout()
plt.show()

```





Sur les figures obtenues, la transformée de Fourier 1D de la projection (courbe bleue) se superpose globalement à la tranche correspondante de la transformée de Fourier 2D de l'image (courbe pointillée orange), ce qui confirme le théorème de la tranche centrale. Cependant, on observe que la tranche extraite de la transformée de Fourier 2D est légèrement différente que la transformée 1D de la projection. En effet, la projection pour un seul angle correspond à une intégration qui réduit la dimension du signal et entraîne une perte d'informations provenant des structures situées légèrement à droite et à gauche de cette direction. La transformée 2D reste donc plus complète et plus précise puisqu'elle conserve l'information fréquentielle dans toutes les directions.

Section 6 - Analyse du bruit (3pts)

6.1. Quel est le type de bruit observé en imagerie par rayon X ? Quelle est l'origine de ce bruit ? (0.5pt)

On ignore ici le bruit électronique au niveau des détecteurs.

Le bruit observé en imagerie par rayons X est principalement un bruit de Poisson (bruit quantique). Il provient du fait que les rayons arrivent par événements discrets dans le temps, et que le nombre de photons détectés (N_b) suit un processus de Poisson, pour lequel la variance est égale à la moyenne ($\sigma^2 = \mu$). Ainsi, comme N_i est un processus de Poisson, les fluctuations statistiques du nombre de photons reçus sont à l'origine du bruit dans l'image.

6.2. Simuler 1000 fois la détection d'un capteur qui reçoit en moyenne 5 photons (utiliser une fonction de distribution aléatoire pour générer un vecteur de 1000 éléments). Tracer l'histogramme des valeurs mesurées par ce capteur (fonction "plt.hist").

Refaites l'expérience avec 10, 50, et 1000 photons reçus. (0.5pt)

La fonction [numpy.random.poisson]

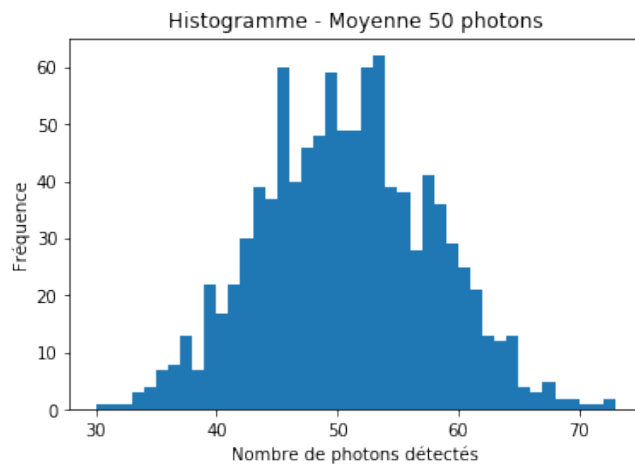
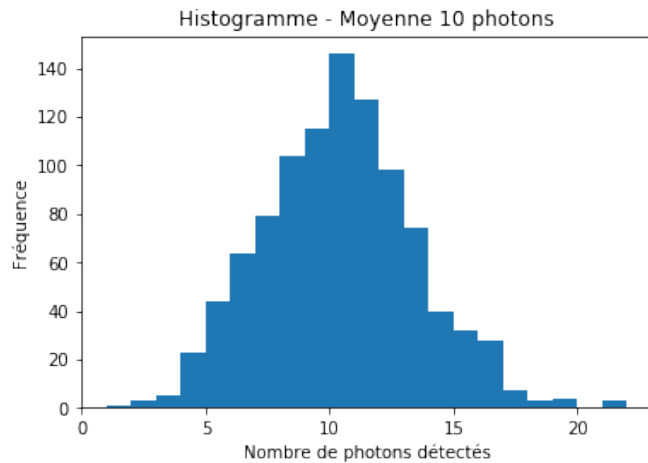
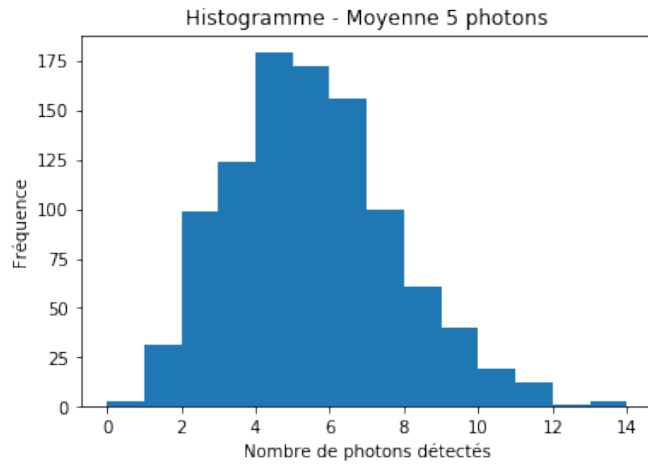
(<https://numpy.org/doc/stable/reference/random/generated/numpy.random.poisson.html>) vous sera utile.

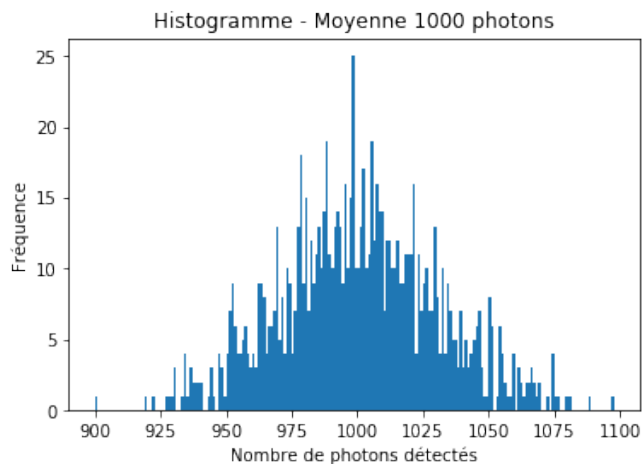
```
In [3]: nb_photons = [5, 10, 50, 1000]

# >>>> Commencez votre code ici <<<<
for lam in nb_photons:
```

```
# Génère 1000 valeurs suivant une loi de Poisson
data = np.random.poisson(lam=lam, size=1000)

#générer la figure
plt.figure()
plt.hist(data, bins=range(min(data), max(data)+2))
plt.title(f"Histogramme : Moyenne {lam} photons")
plt.xlabel("Nombre de photons détectés")
plt.ylabel("Fréquence")
plt.show()
```





6.3. Simulez le bruit obtenu lors de l'acquisition du signal des sections précédentes (attention à placer le bruit au bon endroit pour modéliser le phénomène physique en cause). Montrez la rétroprojection filtrée ainsi obtenue. (1pt)

Vous pouvez utiliser le filtrage de la fonction `iradon`.

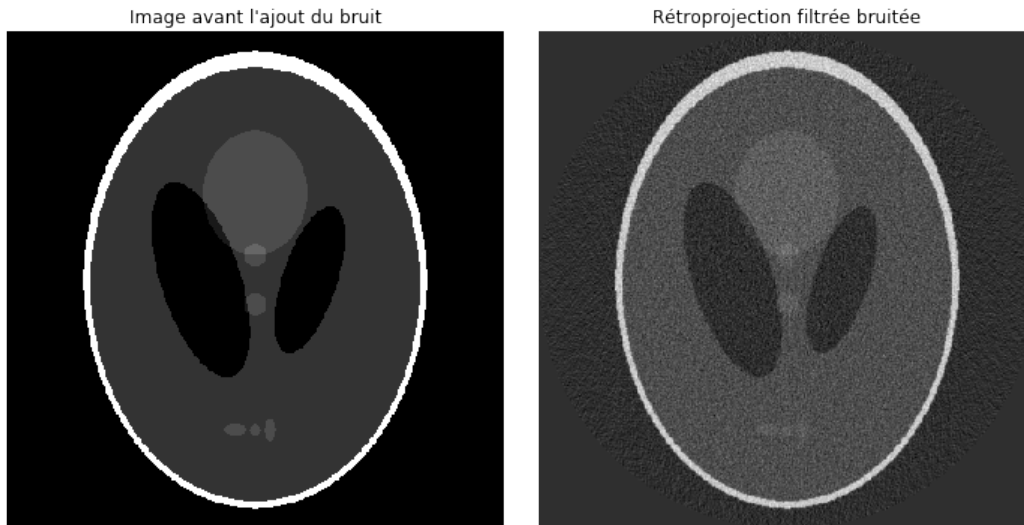
In [65]:

```
# >>>> Commencez votre code ici <<<<
n_pixels = image.shape[0] # Image test phantom
angles = np.linspace(0., 180., max(image.shape), endpoint=False) # Définir les

# Calculer les projections idéales (sans bruit)
sinogram_ideal = radon(image, theta=angles, circle=True)

# Simuler le bruit de Poisson, on suppose que le signal est proportionnel au
mean_photons = 10 # peut changer 5, 10, 50, 1000, j'ai choisi 10
sinogram_noisy = np.random.poisson(sinogram_ideal * mean_photons) / mean_photons
# Reconstruction par rétroprojection filtrée
reconstruction_fbp = iradon(sinogram_noisy, theta=angles, filter_name='ramp',

# générer les graphiques avant et après bruit
plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
plt.imshow(image, cmap='gray')
plt.title("Image avant l'ajout du bruit")
plt.axis('off')
plt.subplot(1,2,2)
plt.imshow(reconstruction_fbp, cmap='gray')
plt.title("Rétroprojection filtrée bruitée")
plt.axis('off')
plt.tight_layout()
plt.show()
```

6.4. Refaire la même expérience avec une quantité de radiations 10 fois plus grande. Qu'observez-vous ? (0.5pt)

```
In [66]: # >>>> Commencez votre code ici <<<<
# 10x plus de radiations (mêmes variables que 6.3)
mean_photons_10x = 10 * mean_photons

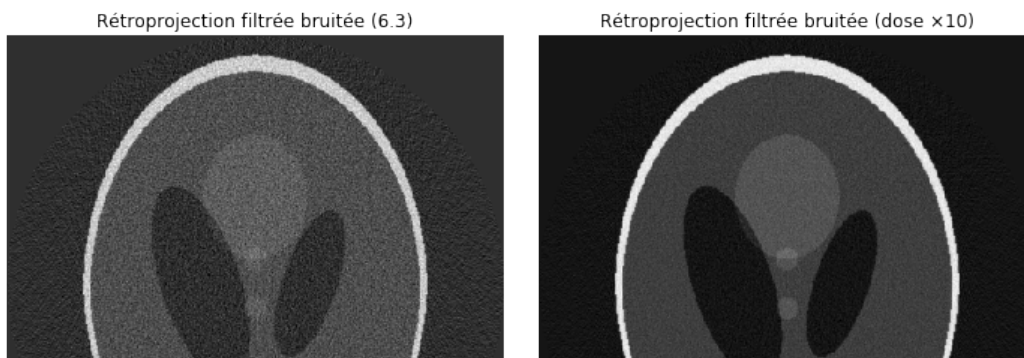
sinogram_noisy_10x = np.random.poisson(sinogram_ideal * mean_photons_10x) / m
reconstruction_fbp_10x = iradon(sinogram_noisy_10x, theta=angles, filter_name=

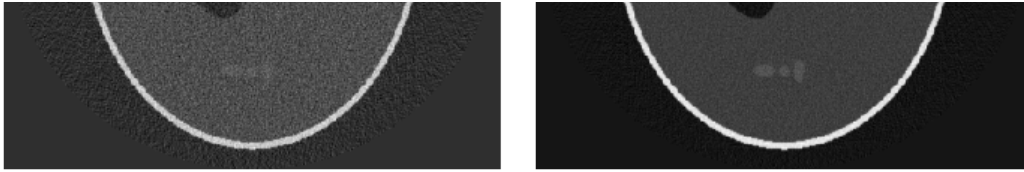
# Affichage à côté
plt.figure(figsize=(10,5))

plt.subplot(1,2,1)
plt.imshow(reconstruction_fbp, cmap='gray')
plt.title("Rétroprojection filtrée bruitée (6.3)")
plt.axis('off')

plt.subplot(1,2,2)
plt.imshow(reconstruction_fbp_10x, cmap='gray')
plt.title("Rétroprojection filtrée bruitée (dose x10)")
plt.axis('off')

plt.tight_layout()
plt.show()
```





Avec une quantité de radiations 10 fois plus grande, l'image reconstruite apparaît nettement moins bruitée et plus homogène. Les contours restent bien définis, mais le bruit amplifié par le filtre rampe est beaucoup moins visible. Cela est cohérent avec le fait que, pour un bruit de Poisson, le rapport signal sur bruit augmente lorsque le nombre de photons détectés augmente.

6.5. Sur le signal bruité, pour une même quantité de radiations totale, comparer des scans acquis:

- avec un grand nombre d'angles et une petite quantité de radiations pour chaque angle.
- avec un petit nombre d'angles et une grande quantité de radiations pour chaque angle.

Montrez les résultats obtenus pour 4, 16, 90 et 180 angles. Quel choix vous semble être le meilleur d'un point de vue clinique? (0.5pt)

Astuce: Pour conserver la même quantité de radiations entre toutes les rétroprojections, il faut que la somme des intensités de chaque sinogramme soit égale au nombre de photons reçus lors du scan.

In [74]:

```
# >>>> Commencez votre code ici <<<<

# Dose totale fixée (même quantité totale de photons sur tout le scan)
I0_total = mean_photons * len(angles) # dose totale = photons/angle * nb_an
#fonction pour bruite et reconstruire
def bruite_et_reconstruire(sino_ref, theta_ref, n_angles, I0_total):
    # Sous-échantillonnage d'angles (choix d'indices répartis sur 0-180)
    idx = np.round(np.linspace(0, len(theta_ref)-1, n_angles, endpoint=False))
    theta = np.array(theta_ref)[idx]
    sino = sino_ref[:, idx]

    # Dose par angle ajustée pour garder la dose totale constante
    I0_par_angle = I0_total / n_angles

    # Bruit Poisson au détecteur: Id = I0_par_angle * exp(-p)
    Id_ideal = I0_par_angle * np.exp(-sino)
    Id_noisy = np.random.poisson(Id_ideal)
    Id_noisy = np.maximum(Id_noisy, 1) # pour éviter log(0)

    sino_bruite = -np.log(Id_noisy / I0_par_angle) # Retour au sinogramme mes
    retro_bruite = iradon(sino_bruite, theta=theta, filter_name='ramp', circle

    return sino_bruite, retro_bruite

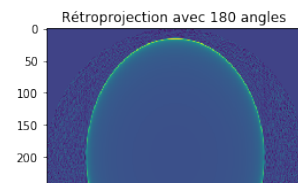
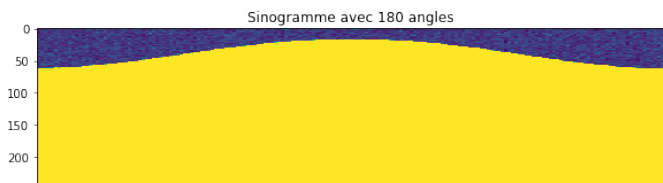
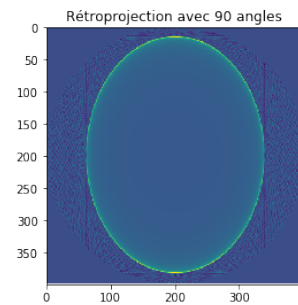
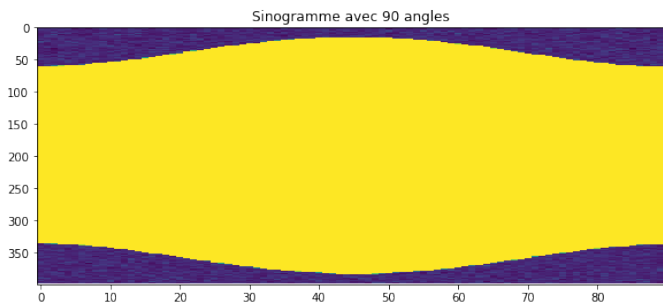
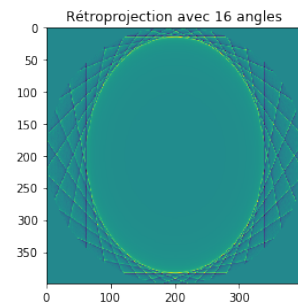
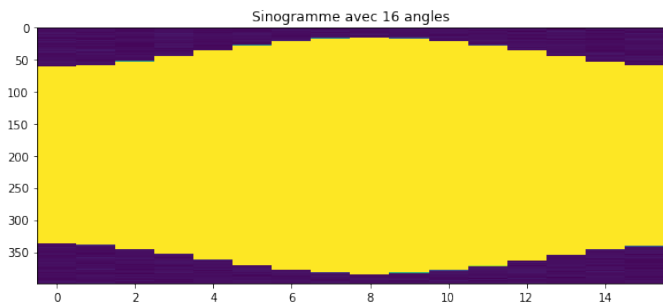
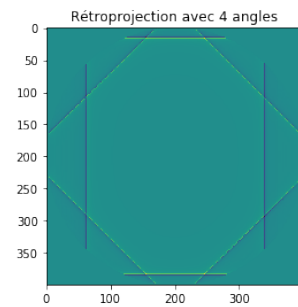
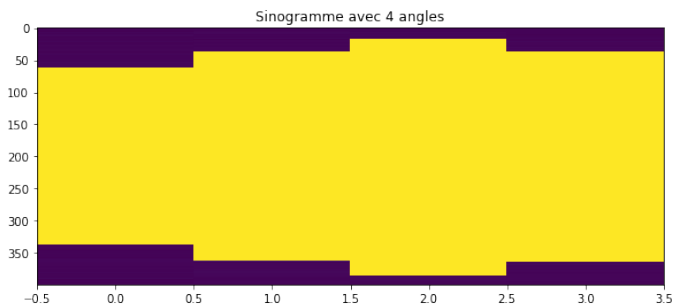
sino4_bruite, retro4_bruite = bruite_et_reconstruire(sinogram_ideal, ang
sino16_bruite, retro16_bruite = bruite_et_reconstruire(sinogram_ideal, ang
```

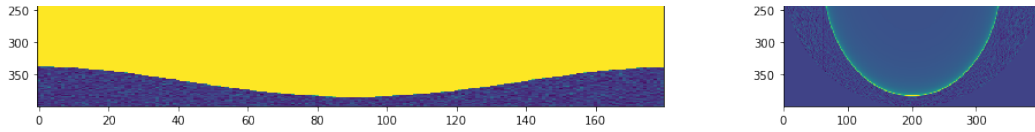
```

sino180_bruite, retro180_bruite = bruite_et_reconstruire(sinogram_ideal, ang
sino90_bruite, retro90_bruite = bruite_et_reconstruire(sinogram_ideal, ang
sino180_bruite, retro180_bruite = bruite_et_reconstruire(sinogram_ideal, ang

# Affichage
plt.figure(figsize=(15,15))
plt.subplot(421); plt.imshow(sino4_bruite,aspect='auto'); plt.title(f'Sinogramme avec 4 angles')
plt.subplot(422); plt.imshow(retro4_bruite,); plt.title(f'Rétroprojection avec 4 angles')
plt.subplot(423); plt.imshow(sino16_bruite,aspect='auto'); plt.title(f'Sinogramme avec 16 angles')
plt.subplot(424); plt.imshow(retro16_bruite,); plt.title(f'Rétroprojection avec 16 angles')
plt.subplot(425); plt.imshow(sino90_bruite,aspect='auto'); plt.title(f'Sinogramme avec 90 angles')
plt.subplot(426); plt.imshow(retro90_bruite,); plt.title(f'Rétroprojection avec 90 angles')
plt.subplot(427); plt.imshow(sino180_bruite,aspect='auto'); plt.title(f'Sinogramme avec 180 angles')
plt.subplot(428); plt.imshow(retro180_bruite,); plt.title(f'Rétroprojection avec 180 angles')
plt.tight_layout()
plt.show()

```





Avec 4 angles, la rétroprojection présente de forts artéfacts en étoile et l'image est très dégradée malgré la forte dose par angle. La structure de l'objet est mal reconstruite. Avec 16 angles, l'image s'améliore, mais des artéfacts directionnels importants persistent. Avec 90 angles, la forme de l'objet est bien définie et les artéfacts sont fortement réduits. Le bruit est plus uniforme. Avec 180 angles, l'image est la plus fidèle : les contours sont bien reconstruits et les artéfacts sont minimisés, même si la dose par angle est plus faible. Alors, à dose totale égale, il est préférable d'utiliser un grand nombre d'angles avec une plus faible quantité de radiations par angle. Cela permet de réduire les artéfacts de reconstruction et d'obtenir une image plus précise et diagnostiquement plus fiable, et ce, même si ça implique une plus grande exposition aux rayons X ionisant pour le patient.